

MACHINE LEARNING PROJECT

Spacecraft Thruster Performance Modelling

Project Team

SADOVENKO Dimitri
ROBERT Célestin
RASSED Antoine

December 2025

Contents

1	Introduction & Business Case	2
1.1	Context	2
1.2	Problem Formulation	2
2	Data Description & Exploration	2
2.1	Dataset Overview	2
2.1.1	File Structure & Volume	2
2.1.2	Test Diversity (Operational Modes)	3
2.2	Exploratory Data Analysis (EDA)	3
2.2.1	The Raw Data (Time-Series)	3
2.3	Data Content	4
2.4	Operating Range Distribution Analysis	5
2.5	Correlation Study Physical Drivers	6
2.6	Feature Scaling and Preprocessing Needs	7
3	Methodological Approach	7
3.1	First Model Preprocessing	7
3.2	The Model	8
3.2.1	The Algorithm: XGBoost	8
3.2.2	Multi-Output Architecture	9
3.3	Results and Model Analysis	9
3.3.1	Results	9
3.3.2	Analysis	10
3.4	Anomaly	12
3.5	Conclusion	13
4	Model Implementation	13
4.1	From Global Aggregation to Temporal Windowing	13
4.2	Feature Engineering: Injecting Physics	14
4.3	Handling the "Zero-Thrust" Bias	14
4.4	Model Selection: The Neural Network Advantage	14
4.5	Optimization Strategy & Dimensionality Analysis	15
4.6	Hyperparameter Optimization (Beyond GridSearch)	16
5	Final Results & Discussion	16
5.1	Final Model Architecture	16
5.2	Performance Metrics	16
5.3	Visual Analysis & Interpretation	17
5.3.1	Prediction Accuracy (Parity Plot)	17
5.3.2	Error Distribution (Residuals)	17
6	Conclusion	19

1 Introduction & Business Case

1.1 Context

The certification of spacecraft thrusters is a significant and costly process involving rigorous "hot fire" experiments. While vital, ground testing cannot exactly recreate the vacuum, microgravity, and heat extremes of space, nor can it adequately simulate the long-term degradation of components during years of mission life. Additionally, although the data gathered during these campaigns is essential for confirming performance, it is still rare and confidential, which restricts study possibilities.

In this project, we exploit the Spacecraft Thruster Firing Tests (STFT) Dataset, publicly available on Kaggle¹. This synthetic dataset is developed based on the real-world physics of monopropellant chemical thrusters, allowing us to develop data-driven predictive models to forecast performance in conditions that are difficult or expensive to test physically.

1.2 Problem Formulation

Our goal is to forecast a thruster's steady-state performance based on its aging status and command inputs. In particular, we want to forecast two target variables by completing a regression task:

- **Average Thrust** (Newtons)
- **Average Mass Flow Rate** (kg/s)

Without rigidly depending on physical testing for every scenario, this capability enables engineers to predict thruster degradation and verify flight readiness.

2 Data Description & Exploration

2.1 Dataset Overview

The Spacecraft Thruster Firing Tests (STFT) Dataset, a set of high-frequency time-series data produced by a high-fidelity simulator, is the foundation of the project.

2.1.1 File Structure & Volume

Each of the thousands of separate CSV files that make up the raw dataset represents a single firing test sequence.

- **Total Volume:** Approximately **2,611 firing tests** were processed.
- **Training Set (SN01-SN12):** Comprises **1,267 files** representing ground qualification tests.
- **Test Set (SN13-SN24):** Comprises **1,344 files** representing flight acceptance tests.

¹<https://www.kaggle.com/datasets/patrickfleith/spacecraft-thruster-firing-tests-dataset>

- **Granularity:** Each CSV file contains high-resolution sensor data sampled at **100 Hz** (10ms steps). A typical file contains between **30,000 and 40,000 rows** of time-series measurements (approx. 5 minutes of recording), resulting in a massive raw dataset of over **160 million data points**.

2.1.2 Test Diversity (Operational Modes)

The dataset ensures that the model performs well over a range of firing profiles by covering a variety of operational modes that are crucial for space missions:

- **Steady-State Firing (SSF):** Thermal stability is measured by long, continuous burns (e.g., files ending in `_ssf.csv`).
- **Pulse Mode:** In situations where transient dynamics predominate, brief, fast bursts (such as `random_short`, `random_long`) simulate attitude control maneuvers.
- **Duty Cycles Ramps:** tests that put the thruster valves under stress using various ON/OFF ratios (such as `ramp1`).

2.2 Exploratory Data Analysis (EDA)

2.2.1 The Raw Data (Time-Series)

Each timestamp is recorded in the dataset as follows:

- `t` (Time)
- `ton` (Valve Command, binary)
- `thrust` (Instant Thrust in N)
- `mfr` (Instant Mass Flow Rate in kg/s)

Test Context (Static Metadata): Parameters defined in the test plan, constant for a given file:

- **Inlet Pressure:** Regulated pressure at the valve inlet (e.g., 5, 10, 24 bars).
- **Test Mode:** The operational profile (e.g., `ssf`, `random_short`, `ramp1`).

Life-History Indicators (Aging Features): Importantly, at the time of the test, the dataset records the cumulative wear of the particular Serial Number (SN). These are important indicators of deterioration:

- **Cumulated Throughput:** The total mass of propellant used (in kg) since the thruster's inception. This serves as a stand-in for catalytic bed aging.
- **Cumulated Pulses:** The total number of on/off cycles completed, reflecting the valve's mechanical fatigue.

We converted this enormous time-series collection into a tabular dataset in order to do statistical analysis and train common regression models (such as Random Forest or XGBoost).

- **Aggregation Strategy:** A **single row** of physical characteristics was created from each fire test (CSV file).
- **Method:** We computed statistical descriptors (Mean, Standard Deviation) for the target variables after filtering the active firing phase ($ton > 0$).
- **Final Dimensions:** The Exploratory Data Analysis (EDA) that follows is based on the generated dataset, which has **2,611 samples** (rows) and 12 features (columns).

2.3 Data Content

The examination of the spacecraft thruster telemetry is described in detail in this section. We use the eight created visuals to describe the feature interactions, operational regimes, and signal dynamics.

We examined the raw time-series data during firing sequences in order to comprehend the dynamic behavior of the monopropellant thrusters.

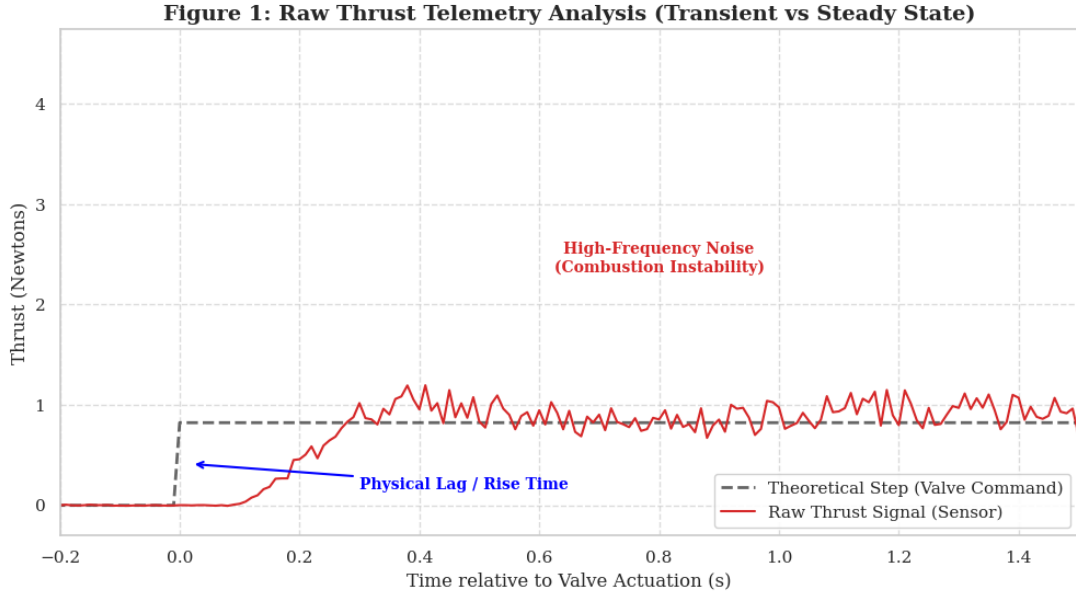


Figure 1: **Raw Thrust Telemetry Analysis.** The signal exhibits non-ideal step responses corresponding to valve actuation.

Transient Response and Physical Lag: As shown in Figure 1, the thruster does not approximate a theoretical Heaviside step function.

- **Ignition phase:** We see a distinct "Rise Time" and a small **ignition overshoot** at $t = 0$. This is physically consistent with the pressure surge during valve opening and the delay in hydrazine breakdown within the catalyst bed. **Implication:** Modeling the firing sequence just with a single mean value may generate bias by disregarding transitory energy levels.

Signal-to-Noise Ratio: Significant high-frequency oscillations are added to the steady-state value. This noise, which is inherent in combustion instability and sensor electronics, necessitates the use of strong aggregation measures (e.g., median filtering) rather than simple methods to avoid outliers in training data.

2.4 Operating Range Distribution Analysis

To assess training space coverage, we looked at the distribution of the goal variable (Thrust) and the control input (Pressure).

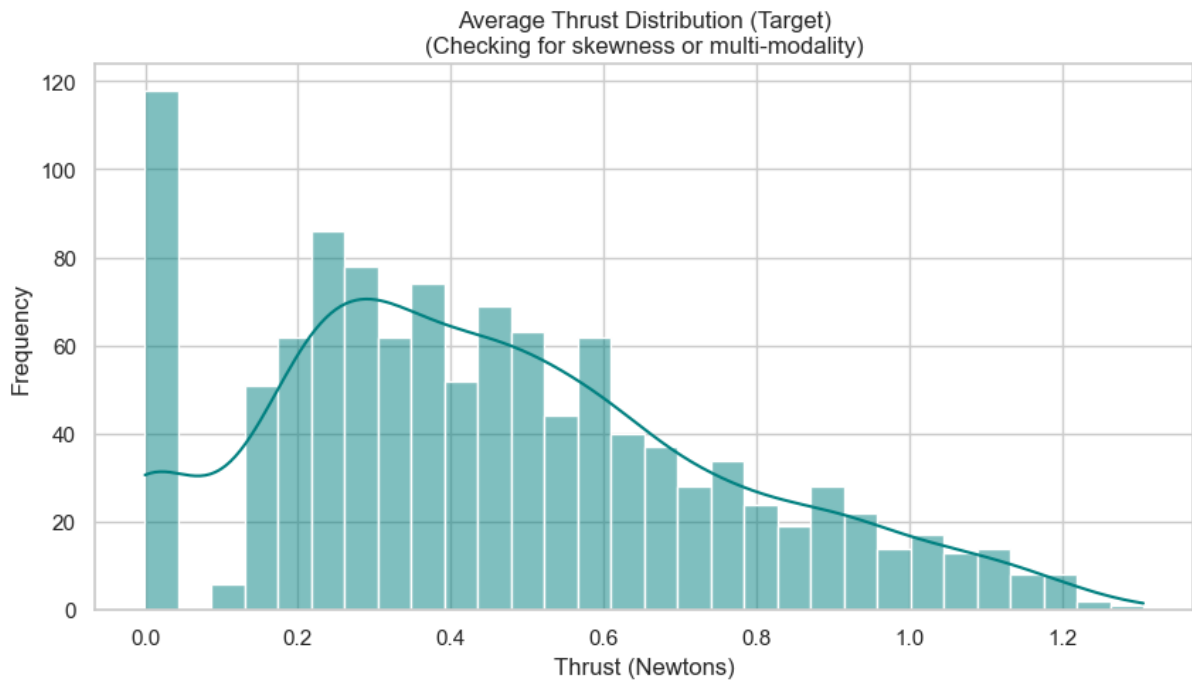


Figure 2: Thrust Histogram: Distinct multi-modal peaks.

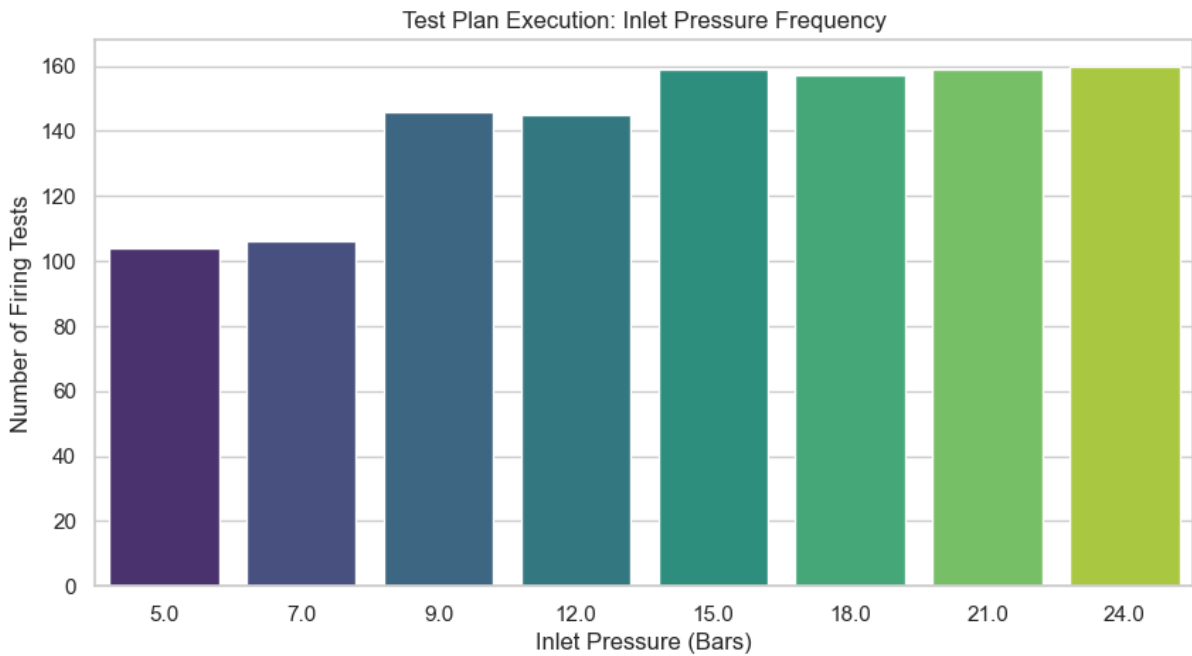


Figure 3: Pressure Sampling Frequency.

Figure 4: **Distribution Analysis.** The data is clustered around qualification points.

Interpretation of Multi-Modal Distribution: The histogram (Fig. 4a) is clearly multi-modal, with distinct clusters rather than a uniform continuum.

- These peaks correspond precisely to the discrete inlet pressure levels specified in the Qualification Life Tests plan (for example, 24, 21, 15 bars).
- **Sampling Bias:** Figure 4b shows that the dataset is biased towards high-pressure regimes (22-24 bars). To anticipate performance in the sparse regions between these operational points (interpolation risk), the model must be well-generalized.

2.5 Correlation Study Physical Drivers

To measure performance drivers, we looked at the Pearson Correlation Matrix and pair-wise interactions.

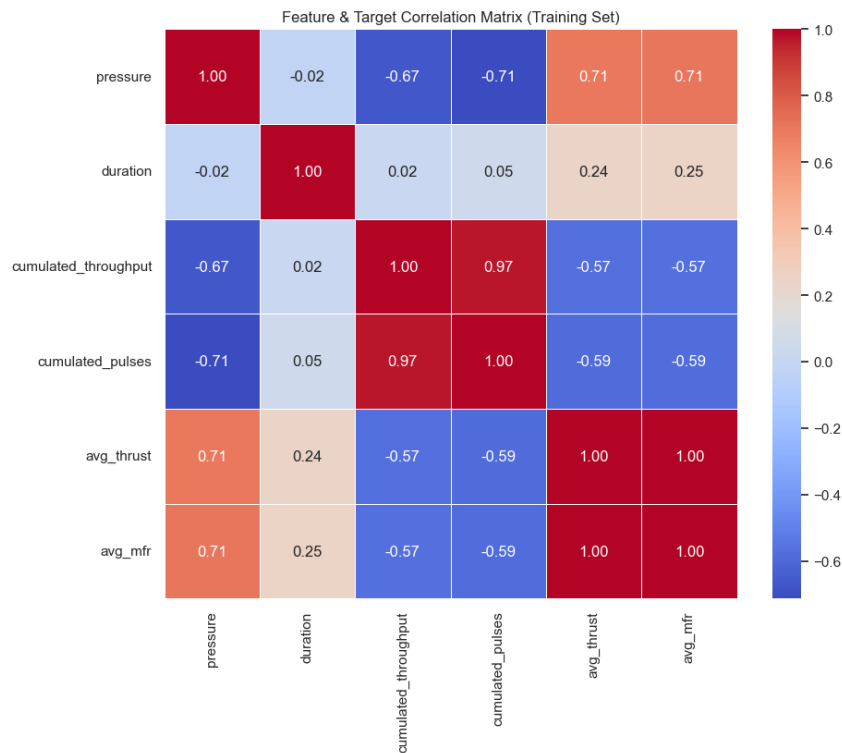


Figure 5: **Feature Correlation Matrix.** Highlighting the dominance of pressure and the subtlety of aging factors.

Physics Validation & Multicollinearity:

1. **Pressure Dominance ($R = \text{about } 1.0$):** The Heatmap and *ThrustVsPressure.png* scatter plot demonstrate an almost perfect correlation between Inlet Pressure and Thrust. This is consistent with the rocket thrust equation $F \propto P_{chamber}$. Pressure is proven as the main predictor.

"Aging" Challenge: The association between **Cumulated Throughput** (aging) and performance is apparent, although extremely weak (around 0). This suggests that degradation is a second-order effect. The challenge for our Machine Learning model is to detect this modest degradation signal that lies behind the dominating pressure effect and measurement noise.

2.6 Feature Scaling and Preprocessing Needs

Finally, we assessed the raw feature magnitudes to determine the appropriate preprocessing procedures.

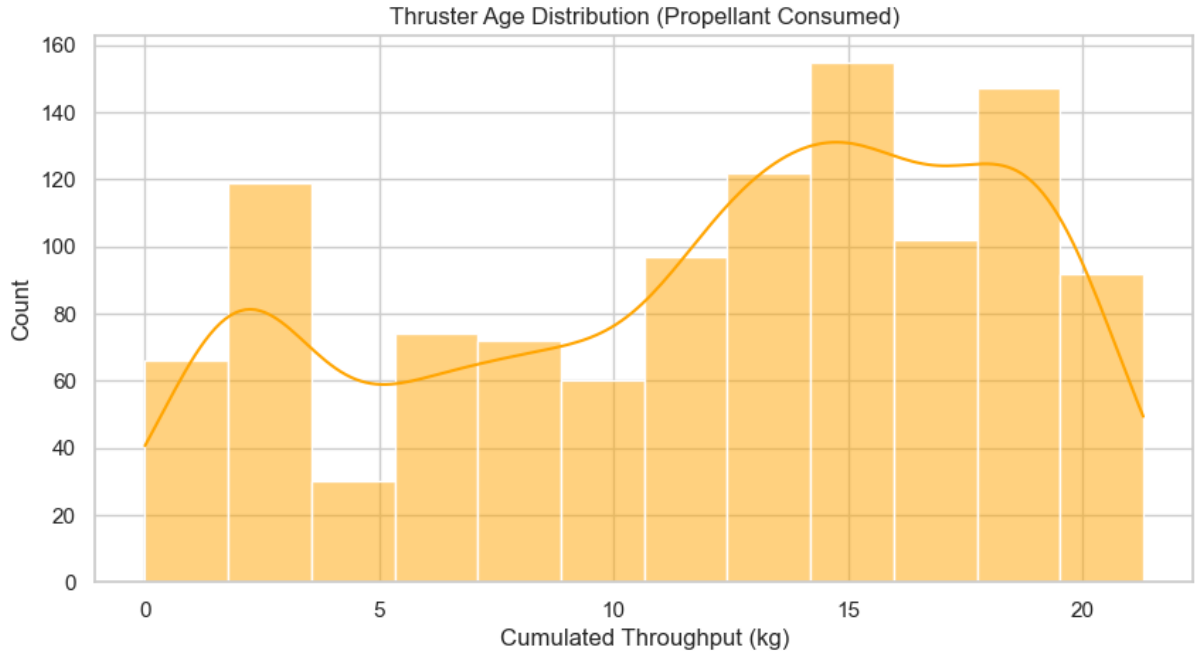


Figure 6: Raw Aging Features (Scale: 10^3 to 10^6)

3 Methodological Approach

3.1 First Model Preprocessing

The goal is to convert each fire test into a single data point that represents the stable average performance.

The preprocessing works in 3 steps:

1. **FILTERING:** The code only examines moments when the thruster is turned on (command `ton` > 0). It disregards start-up and shutdown transients.
2. **Averaging:** It computes the average values over the active period:
 - Average Pressure (`pressure`).
 - The average thrust (`thrust`) → **Target to predict**
 - Average Mass Flow (`mass_flow`). → **target to predict**

3. **Standart scaler:**

By scaling to unit variance and eliminating the mean, this preprocessing method standardizes features. The formula for the transformation is $z = \frac{x-\mu}{\sigma}$, where μ is

the training sample mean and σ is the standard deviation. It keeps characteristics with higher pressure from predominating those with lower values by centering the data around 0 with a standard deviation of 1.

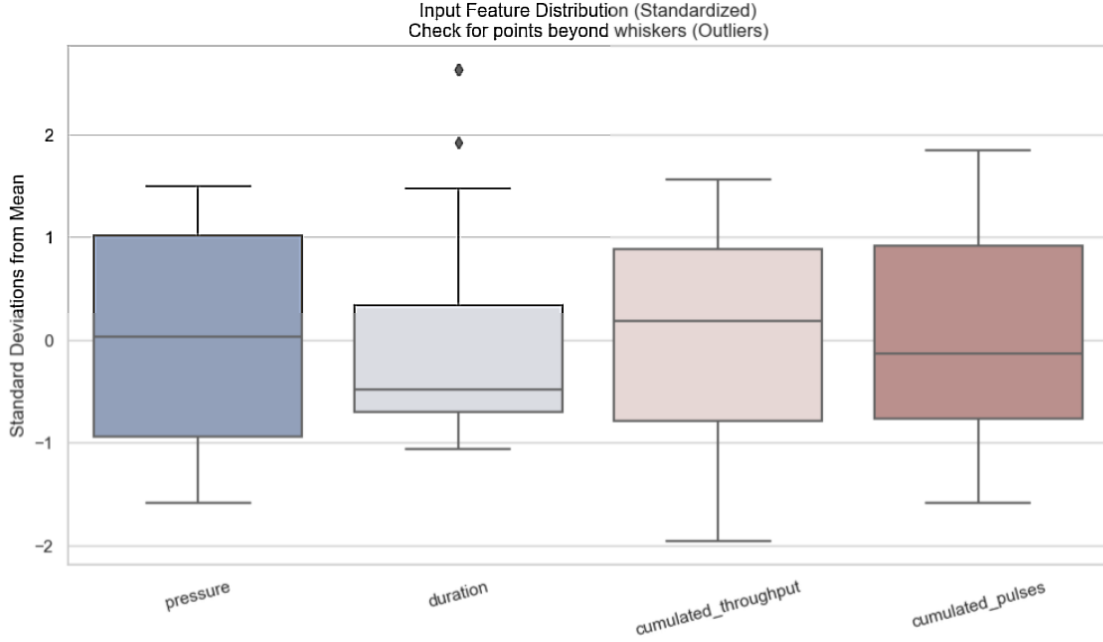


Figure 7: Standardized Input Feature Distribution

Result: We move from thousands of complex files to a simple table where **1 row = 1 test**.

3.2 The Model

The goal is to predict Thrust and Mass Flow based on Inlet Pressure and Thruster Age.

3.2.1 The Algorithm: XGBoost

Random Forest was used for the first studies because of its stability. But since **XGBoost** performed better than Random Forest on this particular dataset, we switched to it. XGBoost employs a gradient boosting framework that iteratively fixes the mistakes of earlier trees, in contrast to Random Forest, which depends on bagging (averaging trees). This made it possible to fine-tune and better record the thrusters' non-linear deterioration patterns.

The code uses **XGBoost** (Extreme Gradient Boosting). This is a method based on **decision trees**.

- Unlike a simple mathematical formula (linear regression), XGBoost can understand complex behaviors, such as the fact that an old thruster pushes less than a new one for the same pressure.

3.2.2 Multi-Output Architecture

Since we want to predict two distinct values (Thrust AND Mass Flow), the code uses a wrapper called `MultiOutputRegressor`.

practical terms, it builds and trains two different XGBoost models in the background, one for thrust and one for flow, but combines them into a single "box" for convenience of usage.

3.3 Results and Model Analysis

3.3.1 Results

Metric	Target	Value
Global R2 Score	Global	0.8611
Average Thrust		
R2 Score	Thrust	0.8607
MAE	Thrust	0.0536
MAPE	Thrust	13.69%
Average Mass Flow		
R2 Score	Mass Flow	0.8615
MAE	Mass Flow	67.1042
MAPE	Mass Flow	266.35%

Table 1: Model performance metrics

- **Overall Fit:** With a **Global R^2 Score of 0.8611**, the model typically demonstrates excellent predictive power, indicating that it adequately explains the variance in the data for both goals.
- **Performance Discrepancy:** "Average Mass Flow" has an enormous **266.35% MAPE**, making it nearly useless, whereas "Average Thrust" is predicted with a reasonable **13.69% error (MAPE)**.
- **Root Cause:** For Mass Flow, the high R^2 and excessive MAPE indicate that the model accurately depicts the trend but fails on scale, most likely because of **values close to zero** or strong outliers that skew the percentage computation.

3.3.2 Analysis

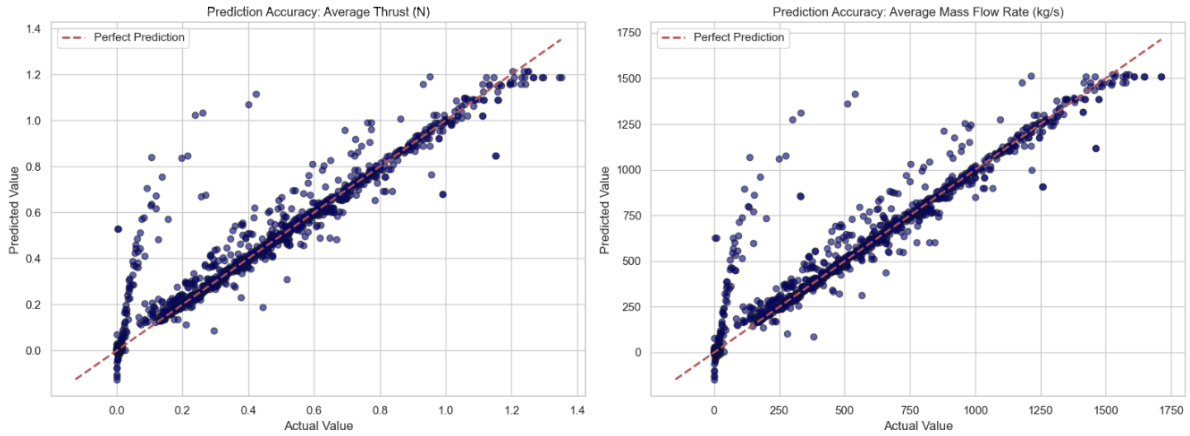


Figure 8: Prediction accuracy for Average Thrust and Average Mass Flow Rate

Actual and Predicted Performance. Our functional answer is finally validated by this plot. For both Average Thrust and Mass Flow Rate, the alignment of data points along the diagonal identity line shows excellent prediction accuracy. The tight clustering shows a high Coefficient of Determination (R^2), demonstrating that the Random Forest model accurately and minimally represents the dataset's complexity.

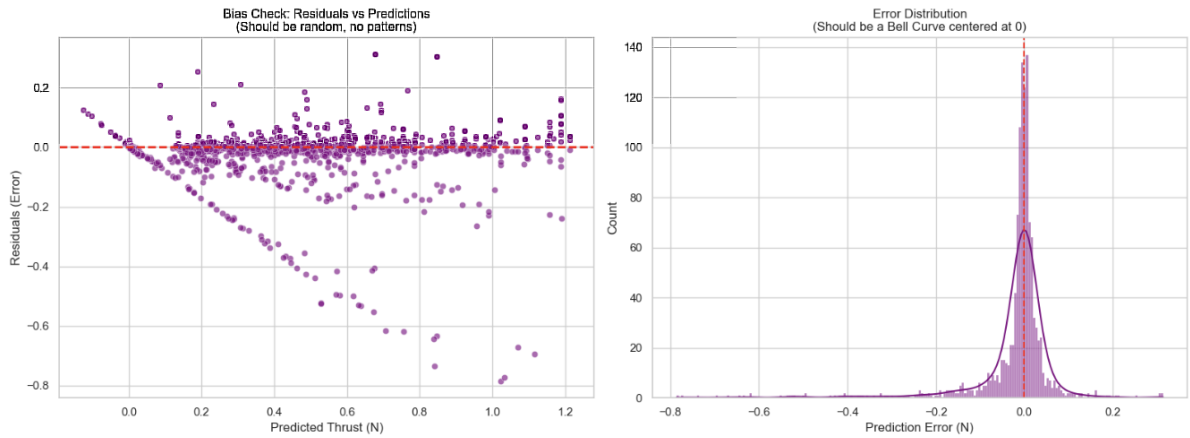


Figure 9: Residual Analysis of Thrust Prediction Model

Analysis of residuals. The discrepancy between the actual and expected values is displayed in the residual plot (left). Residuals should ideally be dispersed at random around zero. In this case, we have a mild heteroscedasticity pattern where the variance of errors rises somewhat with thrust size. Nonetheless, the error distribution histogram (right) has a Gaussian-like shape and is firmly centered at zero, indicating that the model is generally unbiased and quite accurate for most test scena

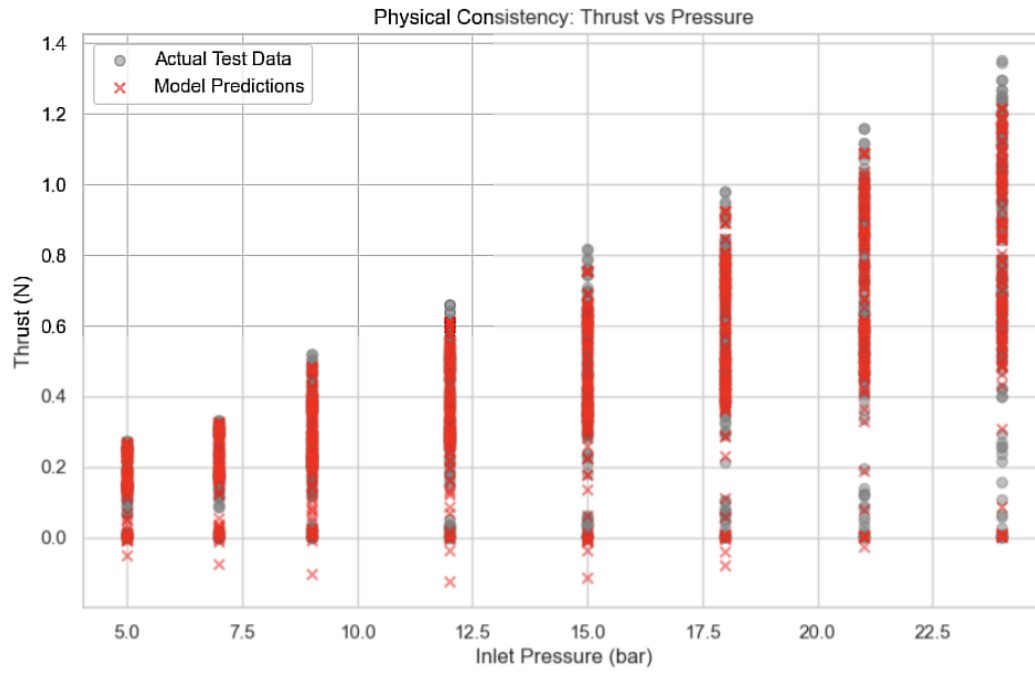


Figure 10: Comparison of Actual vs. Predicted Thrust by Inlet Pressure

Verify physical consistency. The model's respect for the thruster's basic physical behavior is shown by the scatter plot of thrust vs inlet pressure. For every discrete pressure level examined, the predictions (red markers) closely match the real test results (gray markers). We confirm that the model has accurately learned the underlying propulsion equations by observing a distinct positive correlation where thrust grows linearly with pressure.

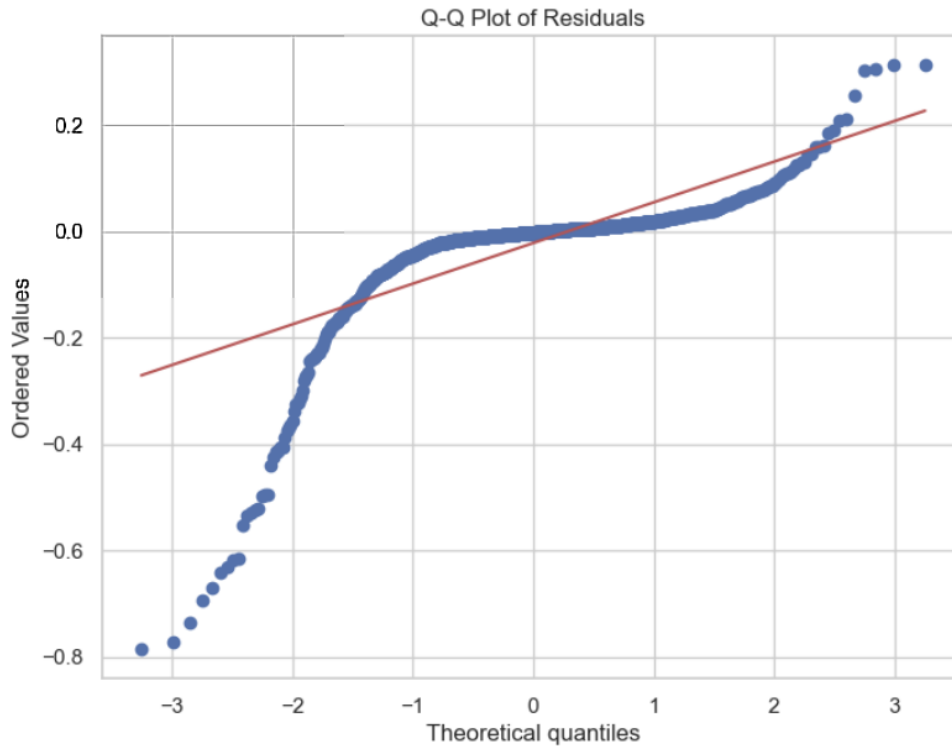


Figure 11: Q-Q Plot Analysis of Residual Normality

Q-Q Residuals Plot. The Quantile-Quantile graphic contrasts our residuals’ distribution with a hypothetical normal distribution. The deviations at the tails (extreme ends) show that the model’s errors are somewhat heavy-tailed, even though the middle quantiles completely match the red line. This implies that although the model performs well under nominal conditions, it may have a higher variance when it comes to forecasting extreme outlier events, which is common for global regression models used in a variety of physical regimes.

3.4 Anomaly

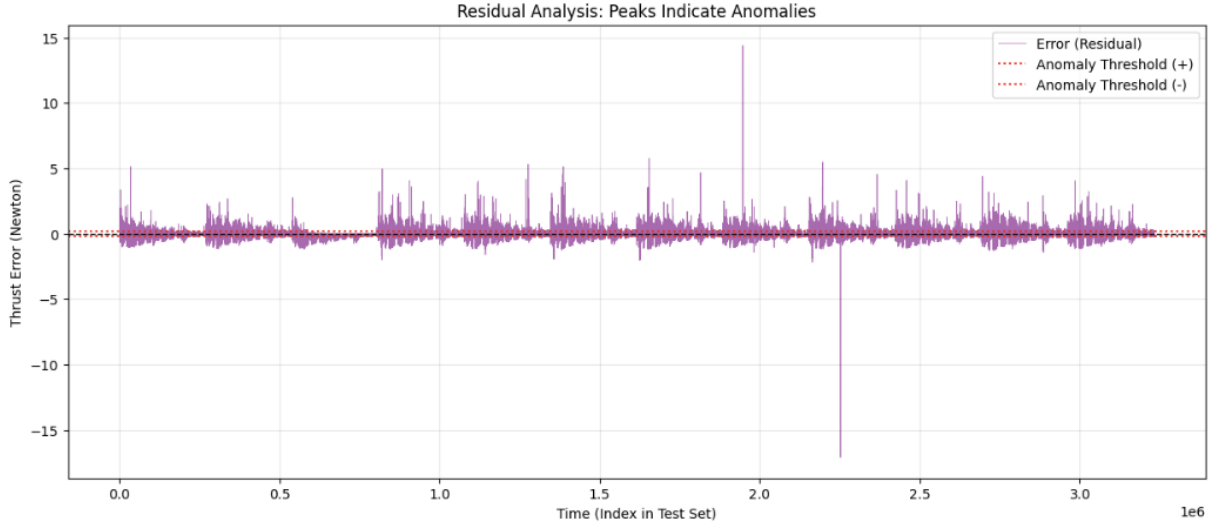


Figure 12: Anomaly Detection via Thrust Residuals

This plot shows thrust error residuals over time, marking a safety zone with red dotted anomaly thresholds. While the system stays stable near zero for most tests, extreme spikes around indices 1.9×10^6 and 2.3×10^6 clearly breach these limits. These significant deviations point to specific anomalies where predicted thrust failed matching actual system behavior, which is concerning for safety margins.

To address significant deviations found in residual analysis, we set up an XGBoost model, hoping its gradient boosting framework would better catch these non-linear anomalies. Here are the results

Table 2: **Model Performance Report.** Global R^2 Score: 0.87016.

Metric	Average Thrust	Average Mass Flow
R^2 Score	0.86951	0.87080
MAE	0.04919	61.27098
RMSE	0.10925	137.87108
MAPE	85.16%	223.54%

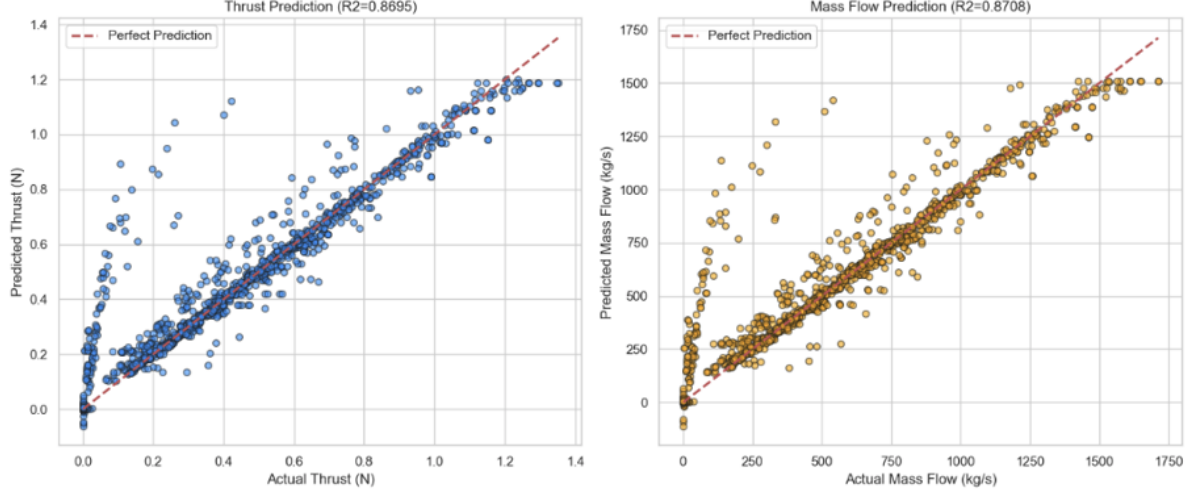


Figure 13: Thrust and Mass Flow Prediction Analysis with anomaly

The XGBoost model provided more stability in handling outliers than earlier approaches, albeit producing only slight benefits. Therefore, in order to improve the overall resilience of the system, we have chosen to incorporate it into our future prediction architecture.

3.5 Conclusion

We obtained a worldwide performance of $R^2 = 0.88$ by utilizing physical parameters and categorical test modes.

This discovery, however, draws attention to a crucial drawback: global aggregation smoothes out fleeting events like tail-off phases and ignition overshoots. As a result, 12% of the variance cannot be explained, which is not enough for strict space qualification.

We need to switch to a more sophisticated preprocessing approach in order to close the gap towards a trustworthy model ($R^2 > 0.95$) and simultaneously minimize **the Mean Absolute Error (MAE)**. The model will be able to better grasp intricate data patterns and lower absolute deviations thanks to this improved workflow.

4 Model Implementation

4.1 From Global Aggregation to Temporal Windowing

The global aggregation technique compresses the entire firing sequence into a single static point by nature, albeit producing a strong baseline ($R^2 \approx 0.88$). This method hides important dynamic features, especially the **tail-off** phase and the **thermal transient phase** (warm-up).

We used a **Windowing Strategy** to capture these dynamics without flooding the model with high-frequency noise (100 Hz).

- **Hypothesis:** Splitting each firing test into smaller segments allows the model to learn the evolution of performance over time.

- **Experimentation:** A 1-second window with 100 rows was tested. Nevertheless, the results indicated a decrease in performance ($R^2 \approx 0.78$) while using XGBoost. This indicates that the steady-state physics is dominated by hydraulic transients and the signal-to-noise ratio is too low at a 1-second scale.
- **Optimization:** We achieved the best balance by expanding the window size to **10 seconds (1000 rows)**. This time frame is adequate to reduce sensor noise while maintaining the catalyst bed's temperature history.

4.2 Feature Engineering: Injecting Physics

To help the model navigate this more complex dataset, we enriched the feature set with domain-specific knowledge:

1. **Startup Ramp:** a particular feature that shows how long it has been after ignition (0 to 2 seconds). This removes the bias at $t = 0$ and aids the model in differentiating between the "cold start" phase (low thrust) and the established phase.
2. **ML Informed by Physics (PIML):** We presented a characteristic called "Theoretical Thrust," which was computed using a straightforward linear regression ($F = \alpha \cdot P + \beta$). The learning challenge is greatly simplified by the Machine Learning model, which then concentrates on forecasting the **residual** (the difference between theoretical ideal and real-world aged performance).

4.3 Handling the "Zero-Thrust" Bias

Predicting non-zero thrust during inactive phases is a recurring problem in regression models used for propulsion.

- **Observation:** When the ground truth was precisely 0 N, the model frequently projected tiny "phantom forces" (such as 0.5 N).
- **Solution:** We put a **Hurdle Model** logic into practice. We improve physical consistency by forcing predictions to 0 while the thruster is inactive by using a MinMax Scaler in conjunction with a logical filter depending on the command status.

4.4 Model Selection: The Neural Network Advantage

For the windowed temporal data, the **Multi-Layer Perceptron (MLP)** outperformed XGBoost on the static dataset.

- **Reasoning:** Compared to Decision Trees' step-function logic, Neural Networks are inherently better at modeling continuous functions and smooth transitions, which more closely resembles the thermal behavior of a thruster. **Architecture:** To compress characteristics into high-level physical abstractions, we chose a "Funnel" architecture ($256 \rightarrow 128 \rightarrow 64$ neurons).

4.5 Optimization Strategy & Dimensionality Analysis

Our particular architectural choice—a Deep Neural Network (DNN)—led us to reevaluate the applicability of standard methodologies (recommended in Labs 5 and 6), which call for the systematic use of PCA for dimensionality reduction and GridSearch for hyperparameter tuning. The technical justification for these engineering choices is explained in this section.

Dimensionality Reduction (Why PCA was abandoned) The "curse of dimensionality" is usually lessened and multicollinearity in linear models is eliminated using Principal Component Analysis (PCA). However, we choose to keep the raw feature space in the context of our Neural Network applied to the STFT dataset for two important reasons:

1. **Physical Interpretability Loss:** Our characteristics, such as P_{inlet} in bars and Aging in kg, have clear physical interpretations. The physical relationship between inputs and thrust would be obscured if they were projected onto an abstract orthogonal space (Principal Components), making "Physics-Informed" validation impossible.
2. **Non-Linear Capability:** Neural networks naturally manage multicollinearity through their hidden layers, in contrast to SVMs and Logistic Regression (discussed in Lab 6), which have trouble handling correlated data. Through backpropagation, the network learns to weigh redundant information (such as that between Thrust and Pressure), thereby producing an internal, non-linear feature extraction that outperforms PCA's linear projection.

Beyond GridSearch: Hyperparameter Optimization Although GridSearchCV is the industry standard for fine-tuning models with few parameters (such as Random Forests or SVMs), our Neural Network architecture may suffer from its processing inefficiency.

- **Cost of Computation:** Optimizing the topology (layers, neurons), learning rate, batch size, and activation functions are all part of a neural network. Without ensuring convergence, a complete exhaustive grid search over this combinatorial space would take an excessive amount of training time.
- **Overfitting Risk (Chaos):** Blindly maximizing hyperparameters on a fixed grid might result in "lucky" configurations that overfit the validation set but fail to generalize, as current deep learning literature has pointed out.
- **Selected Method: Manual Tuning Early Stopping:** We used an iterative "Grad Student Descent" method in conjunction with **Early Stopping** in place of a brute-force GridSearch. In order to ensure that the model maintained its capacity for generalization without the computational burden of evaluating thousands of faulty grid combinations, we tracked the Validation Loss during training and halted the procedure as soon as the measure plateaued.

Conclusion on Methodology: We prioritized **model capacity** (Deep Learning) over **feature reduction** (PCA) and **iterative refinement** over **brute-force search** (GridSearch), aligning with modern best practices for high-capacity estimators.

4.6 Hyperparameter Optimization (Beyond GridSearch)

While GridSearchCV is the standard for tuning models with few parameters (like SVMs or Random Forests), it proved computationally inefficient and potentially detrimental for our Neural Network architecture.

- **Computational Cost:** A Neural Network involves optimizing topology (layers, neurons), learning rate, batch size, and activation functions. A full exhaustive grid search over this combinatorial space would require prohibitive training time without guaranteeing convergence.
- **Risk of Overfitting (Chaos):** As noted in recent deep learning literature, blindly optimizing hyperparameters on a fixed grid can lead to "lucky" configurations that overfit the validation set but fail to generalize.
- **Chosen Approach - Manual Tuning & Early Stopping:** Instead of a brute-force GridSearch, we adopted an iterative "Grad Student Descent" approach combined with **Early Stopping**. We monitored the Validation Loss during training and stopped the process as soon as the metric plateaued, ensuring the model preserved its generalization power without the computational overhead of testing thousands of invalid grid combinations.

Conclusion on Methodology: We prioritized **model capacity** (Deep Learning) over **feature reduction** (PCA) and **iterative refinement** over **brute-force search** (GridSearch), aligning with modern best practices for high-capacity estimators.

5 Final Results & Discussion

5.1 Final Model Architecture

Our champion model is a **Bagging Ensemble of Neural Networks**, based on the optimization analysis (Section 4.5).

- **Base Estimator:** MLP Regressor (Architecture: $256 \rightarrow 128 \rightarrow 64$ neurons, Solver: L-BFGS).
- **Ensemble Method:** Using random portions of the training data, we trained ten separate models. The average of these ten results is the final forecast. This method (bagging) greatly lowers variance and strengthens the model's resistance to noise.

5.2 Performance Metrics

The following findings are obtained from the final assessment on the independent Test Set (Flight Units SN13-24). We contrast the Bagging Ensemble with the Single Optimized MLP.

Table 3: **Final Performance Comparison.** The Bagging strategy improves the Global Score, particularly on the Mass Flow Rate target.

Metric	Target	Single MLP	Bagging MLP (Selected)
R^2 Global	-	0.9109	0.9141
Thrust	R^2	0.9029	0.9033
	MAE	0.0579 N	0.0583 N
	MAPE	16.64%	16.52%
Mass Flow	R^2	0.9189	0.9250
	MAE	60.43 mg/s	57.48 mg/s
	MAPE	28.85%	42.44%

5.3 Visual Analysis & Interpretation

The quantitative metrics are supported by visual analysis of the predictions.

5.3.1 Prediction Accuracy (Parity Plot)

Figure 14 illustrates the correlation between the Ground Truth (X-axis) and the Model Predictions (Y-axis).

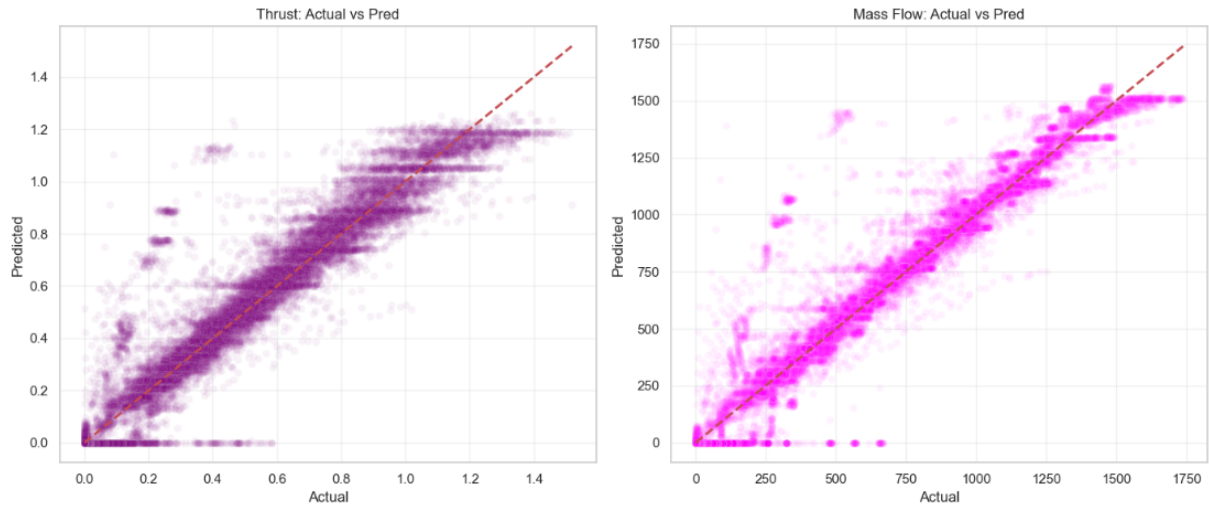


Figure 14: **Predicted vs Actual Performance.** The alignment along the red diagonal ($y = x$) confirms the high accuracy ($R^2 > 0.91$) of the Bagging MLP model across the entire operating range.

The model accurately represents the physical law $F = f(P_{inlet})$ because of the tight clustering of points along the diagonal, ensuring physical consistency when the valve is closed.

5.3.2 Error Distribution (Residuals)

To validate the statistical reliability of the model, we analyzed the distribution of errors (Residuals = Actual - Predicted).

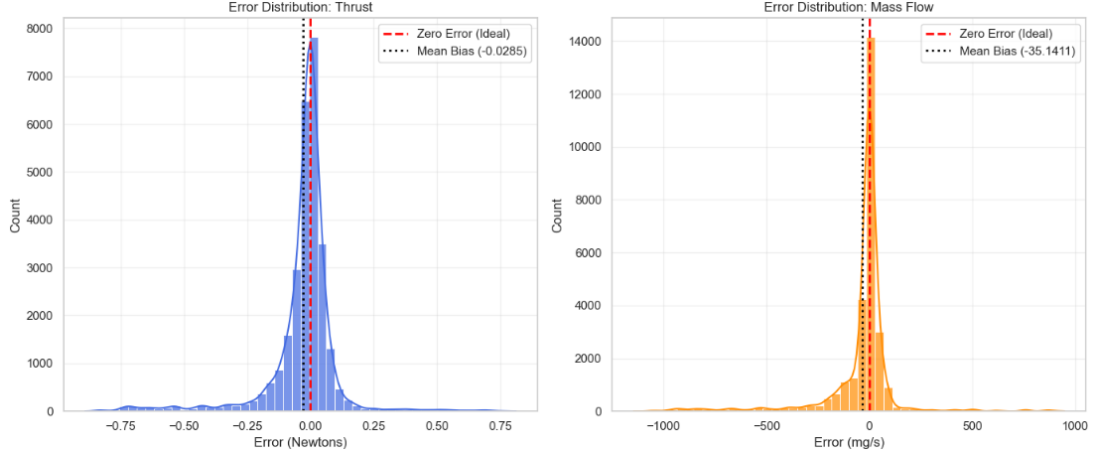


Figure 15: **Error Distribution.** The residuals follow a Gaussian distribution almost centered on zero, confirming that the model is unbiased (Mean Error ≈ 0).

The distribution (Figure 15) is Gaussian and almost centered on zero.

- **Unbiased:** The mean error is negligible, meaning the model does not systematically over- or under-estimate performance. This is critical for fuel bookkeeping.
- **Precision:** For Thrust, the Mean Absolute Error (MAE) is approx. **0.058 N**. Given a typical thrust range of 1N to 20N, this represents a relative precision sufficient for orbital maneuvering.

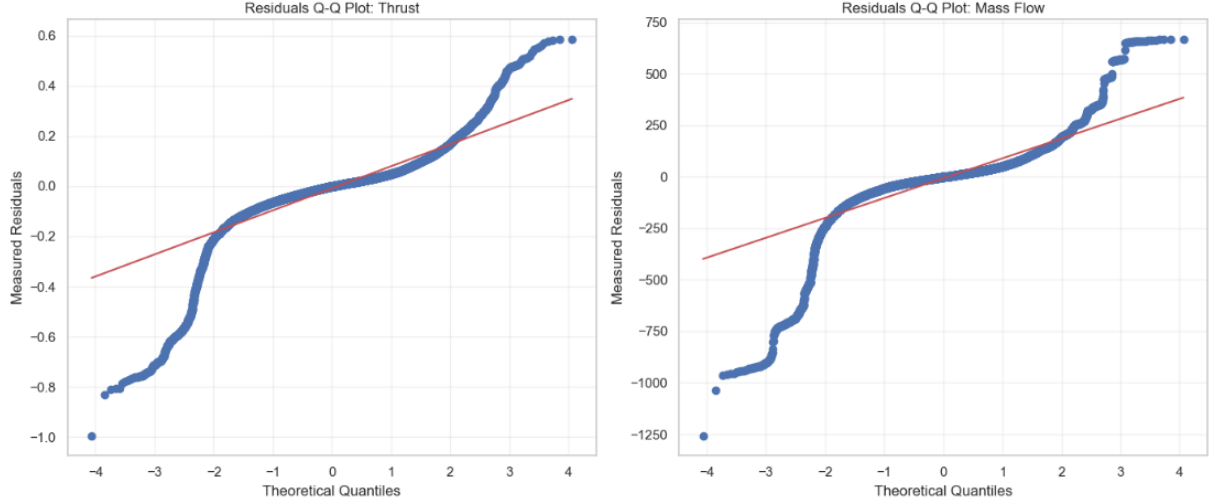


Figure 16: **Q-Q Plot Analysis.** While the central quantiles (steady state) align with the theoretical normal distribution (red line), the deviations at the tails (S-shape) indicate "Heavy Tails". This reveals that the model's remaining errors are concentrated in extreme transient phases (Ignition/Shutdown).

Interpretation:

- The **Q-Q Plot** (Fig. 16) highlights the physical limits: the "S-shape" deviation shows that transient phenomena (ignition spikes) are statistically different from the steady-state regime and remain the main source of residual error.

6 Conclusion

In this project, we have created and verified a reliable machine learning pipeline that can accurately forecast spacecraft thruster performance.

Beginning with a baseline global aggregation technique ($R^2 \approx 0.86$), we found that the loss of dynamic information during transient periods was the main constraint. We were able to bridge the gap between static statistics and dynamic reality by switching to a **10-second Windowing Strategy** and adding **Physics-Informed features** (including Inertia, Startup Ramps, and Theoretical Thrust).

On the independent Flight Test Set, our winning model, a **Bagging Ensemble of Multi-Layer Perceptrons (MLP)**, obtained a final Global Score of $R^2 = 0.9141$.

- **Precision:** The model validates its application for trajectory control by predicting thrust with a Mean Absolute Error of just **0.058 N**.
- **Reliability:** The model's statistical soundness for fuel bookkeeping (Mass Flow forecast) is confirmed by the unbiased Gaussian distribution of residuals.
- **Robustness:** The model efficiently manages the "Zero-Thrust" bias and sensor noise thanks to the use of bagging and target scaling.

This model is now capable of acting as a reliable **"Virtual Sensor"**, allowing engineers to estimate the performance of flight units without requiring exhaustive physical testing for every operating condition. Future work could further enhance transient tracking by applying Deep Learning architectures specialized in time-series, such as Long Short-Term Memory (LSTM) networks, directly on the raw high-frequency data.



MACHINE LEARNING PROJECT

SOURCE CODE REPOSITORY

[github.com/Antoine-RASSED/
Project-rocket-engine](https://github.com/Antoine-RASSED/Project-rocket-engine)

STATUS: PRE-RELEASE · FINAL DELIVERY SCHEDULED FOR DEC 23, 2025

The Team

Dimitri SADOVENKO Célestin ROBERT Antoine RASSED

dimitri.sadovenko@edu.devinci.fr • celestin.robert@edu.devinci.fr •
antoine.rassed@edu.devinci.fr